让大型代码库更容易理解、修改与验证

把一次性的结构重构，
转化为可反复复用的研发效率收益。

Agent 驱动重构 × 行为契约 × 独立验证 × 真实任务评测

01 为什么值得做

02 如何可靠地做

03 如何证明有效

目标是降低后续研发成本，核心能力是可验证的重构闭环

将“代码是否更精简”与“是否仍然正确、是否更易使用”分开衡量。

价值

让每次修改更省力

期望减少重复阅读与定位，
降低接口变更和回归排查的负担。

方法

把重构变成可审查流程

功能范围与目标明确；
原版基线、验证和结果版本绑定。

阶段

闭环已成型，收益待扩证

已有真实项目试验与审查；
下一步补齐验收并跨仓库复现。

管理上的判断依据：功能与性能保持 → 任务成功 → 成本下降 → 可重复复现

依据：main README 与 task-effort；设计分支 refactoring policy

复杂度的代价，会在每一次开发任务中重新支付

项目的价值假设：一次有证据的简化，降低后续多次任务的累计负担。

01

重复实现

相似规则分散在多处，
一次改动需要反复同步。

02

隐式知识

理解接口与配置，
需要跨文件补全前提。

03

验证不透明

检查通过覆盖了什么，
与新代码是否匹配不清楚。

累计净收益 ≈ 后续任务节省 − 重构、验证与维护投入

同一成功标准、完整成本记录、足够多的后续任务，是验证价值的前提。

外部契约保持稳定，内部结构可以重新设计

功能范围先冻结；实现方式可以改变；每个结论都要对应实际证据。

必须保持

所选功能与公开接口
校验、错误说明、日志与边界
类型与配置语义
必要性能和支持的运行路径

可以重新设计

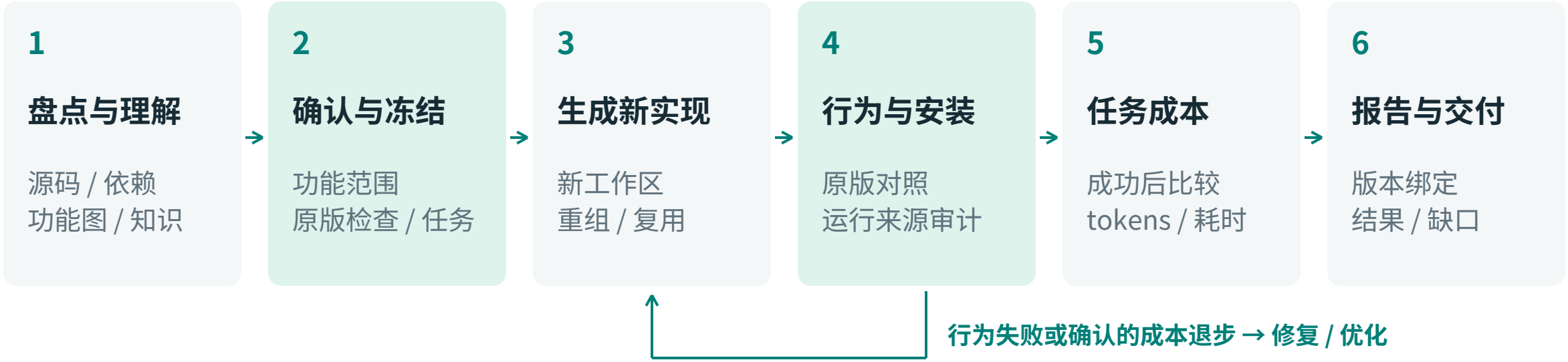
模块组织与内部架构
重复规则与共享实现
向量化和成熟算子复用
冗余说明的精简

范围选择与体积目标分离：调小目标，不会自动减少必须保留的功能。

依据：token 分支 docs/graph-rewrite.md 与 refactoring policy

先明确功能和基线，再重构、验证与交付

所有修复沿用同一功能范围和原始证据；未执行或未知的部分保留为缺口。



交付内容：新实现分支 + 验证证据 + 任务成本结果 + 明确的未覆盖项

依据：main README 工作流程；token 分支 graph-rewrite.md

面向人解释功能，面向 Agent 提供可追溯的行为知识

CLI、MCP 与 Skill 共享同一 Python 核心；实现方与验证方承担不同职责。

理解与审阅

给人：功能图

选择范围、查看依赖

给 Agent：知识包

接口 / 语义 / 来源
全局约束 / 未知项

→

实现 Agent

读取所选行为知识
在新工作区生成实现
依据有限反馈修复

→

独立验证与评测

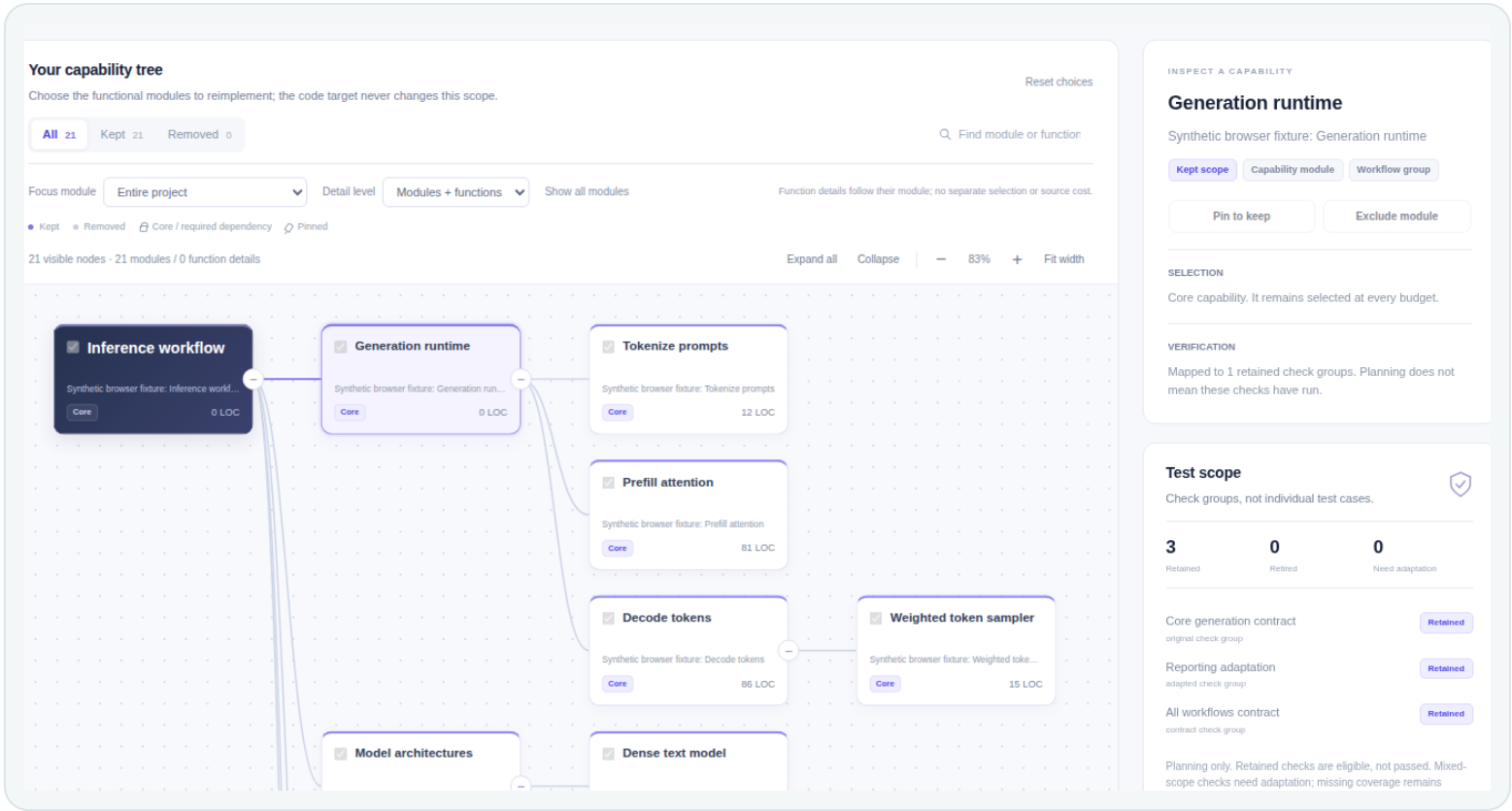
原始测试与行为基线
新环境安装与运行来源
冻结任务与隐藏验收
与候选版本绑定的报告

版本演进：token 分支增加只读原码参考以补全语义；原始测试与隐藏任务仍由验证方保管。

依据：main agent-knowledge / graph-rewrite；token 分支原码参考策略

用户用功能图确认范围，系统负责执行与追踪证据

把仓库级操作转成可理解、可选择、可审阅的功能级工作流。



01 先看功能

按能力组织，而非只看目录。

02 再定范围

核心能力和必要依赖受保护。

03 独立设目标

选择范围与体积目标分别控制。

界面来自 main/docs/capability-planner.png；合成示例，非 Detectron2 实测结果

把“通过了什么”绑定到“哪一版代码”

保留失败和未知，才能让后续评审、修复与交付建立在同一份事实之上。

冻结

先固定原版证据

原版版本、输入、测试、输出与任务先冻结。

对照

由独立验证方检查

行为失败保留；
修复后重跑保留检查。

来源

确认运行的是候选

审计模块来源；
新环境验证构建与安装。

绑定

结果随版本更新

代码变化后重新验证；
历史失败与缺口仍可追溯。

检查通过 = 已运行条件下的证据；完整功能等价与全量性能需要额外覆盖。

依据：main README、reliable-workflows 与 post-nanolize-reports

Detectron2 试验说明了潜力，也推动了验收标准升级

历史候选已显著缩小实现体积；独立审查同时发现未覆盖的语义差异。

实现＋配置 / 运行数据 / 构建输入



−52.6%

体积缩减
不等于逻辑或成本降幅

已有证据与尚未完成的验收

160/160 历史行为检查组通过

23/23 历史部分 GPU 用例通过

任务验收未完成

静态审查发现具体行为差异；候选隐藏验收未运行。

经验：精简幅度、功能正确性与任务成本必须独立验收。

来源：2026.11.04 Detectron2 effort49 独立审查；历史候选 8080c703，非 token 分支结果

设计演进：同时优化源码 tokens 与实现逻辑

codex/token-count-and-logic：体积目标成为软目标；功能与性能约束优先。

优化目标 A

源码 tokens

计量原始生产源码文本。
固定分词口径，记录版本。
反映阅读材料的规模。

优化目标 B

实现逻辑

减少重复规则与维护点。
当前用 Python AST 辅助计量。
由结构解释支撑指标变化。

独立验收维度

真实任务成本

在相同任务和模型下，
比较成功完成任务的 tokens。
同时记录耗时与评测开销。

源码 tokens ≠ 模型调用 tokens；指标改善必须服从行为、类型、配置与性能约束。

用固定任务的原版 / 候选对照，验证是否真的更省力

先确认任务成功，再比较成本；一次低成本结果不足以说明稳定改善。

对照实验保持一致

相同功能范围与任务
固定模型与推理设置
原版 / 候选独立副本
全新会话、真实 usage

1

单次筛选

先确认可完成
避免无效重复

2

重复比较

默认每组 3 次
保留波动与缺口

3

隐藏验收

新输入最终检查
不反复反馈答案

低成本试点：先评测代表性“修改＋执行验证”任务，再扩展到理解、诊断和执行。

依据：main 与 token 分支 docs/task-effort.md；主讲采用 token 分支分阶段评测设计

下一步：用少量代表性仓库，建立可复用的验收样板

从工程闭环，走向稳定收益与可扩展的项目接入方法。

01

补齐验收

已知行为差异
关键性能与环境边界

02

复现收益

2–3 个代表性仓库
固定任务与完整成本

03

形成接入模板

功能知识与检查配方
统一报告与复用流程

建议支持：试点业务 owner / 真实测试与性能基线 / 有上限的评测预算 / 独立验收

附录 A｜三类版本与证据的边界

汇报中的“现有能力”“新设计”和“历史案例”分别来自不同版本。

维度	main · f83bd58	token 分支 · 8b95fb5
体积目标	**物理源码行约束**	**源码总 tokens 软目标**
逻辑目标	以实现与任务结果判断	独立的逻辑缩减目标
作者参考输入	功能契约与私有知识	额外提供只读原码 / 配置
默认任务评测	理解、诊断、修改、执行	代表性修改任务；可配置四类
验收节奏	开发集与隐藏变体	先筛选，再重复，最后隐藏验收

历史案例：Detectron2 effort49 候选 8080c703；其缩减率和测试证据不代表新分支结果。

附录 B｜三个指标，回答三个不同的问题

固定定义、记录覆盖范围，让每次比较可以解释和复核。

源码 tokens

问题：文本体积是否减少？
口径：逐文件 tiktoken / o200k_base 求和，记录版本与覆盖。

Python AST

问题：静态实现逻辑是否缩减？
口径：归一化语法树节点；只覆盖 Python，不外推到 CUDA / C++。

任务 tokens

问题：正确完成任务的模型用量是否下降？
口径：真实输入＋输出及重试；评测开销、耗时另列。

所有口径都保留缺口：缺失 usage、覆盖不全或版本不兼容时，不把未知写成 0。

依据：token_metrics.py、logic_metrics.py、docs/task-effort.md（token 分支）

附录 C｜资料来源与汇报使用说明

截至 2026.11.05 的固定版本；幻灯片中的链接可点击，逐页备注附有来源。

01 项目现有能力与流程

main @ f83bd58 · README / agent-knowledge / task-effort

02 双目标与原码参考设计

codex/token-count-and-logic @ 8b95fb5 · graph-rewrite / metrics

03 Detectron2 真实案例审查

2026.11.04 effort49 audit · 候选 @ 8080c703

04 产品界面图

仓库 capability-planner.png · 合成示例

建议讲述顺序：1–12 页主讲（约 15 分钟）；13–15 页用于版本、指标和证据问答。